PGR111, solutions week 10

Problem 1 – concepts & terminology

Briefly explain the following terminology and database terms:

- Determinant and functional dependency (preferably also with an example)
- First normal form (1NF), including the rule
- Second normal form (2NF), including the rule
- Third normal form (3NF), including the rule
- Boyce-Codd normal form (BCNF), including the rule
- Denormalization

Note: Keep your answers. They might become helpful later in the semester. :-)

Look in the textbook and slides for definition of terms.

Task 2 – Normalization to BCNF

Feel free to collaborate with others on the tasks (but you can do the tasks alone as well). Use pen and paper, Lucidchart or something similar for modelling.

Given two tables: CD + Track. A CD can contain multiple tracks, but a track can only belong to one CD. *Important, a couple constraints for this task:* 1) A CD can only contain one artist or a single group. (We cannot have CDs with a mix of several artists/groups). 2) A track can only have a single author.

The current database structure is as follows: *(underscore means PK, asterisk (*) means FK)*
Table CD: cdnum, cdname, artistnum, artistname, releaseyear, releaselocation, price, playtime
Table Track: cdnum, tracknum, time, artistnum, textauthorid, textauthorname.

a) Figure out a primary key for the Track table.

   cdnum, tracknum (each cd can have track 1, track 2, etc., so tracknum alone is not enough)

b) Determine all functional dependencies (or determinants, if you like), and normalize the tables to BCNF. Remove unneeded columns, if any. Add new ones if needed. Create an ER model based on your tables.

   Table CD:
   - cdnum -> cdname, artistnum, artistname, releaseyear, releaselocation, price, ~~playtime~~ (*note:* playtime can be removed, as it can be calculated from SUM(track.time) for a specific cdnum.)
   - artistnum -> artistname

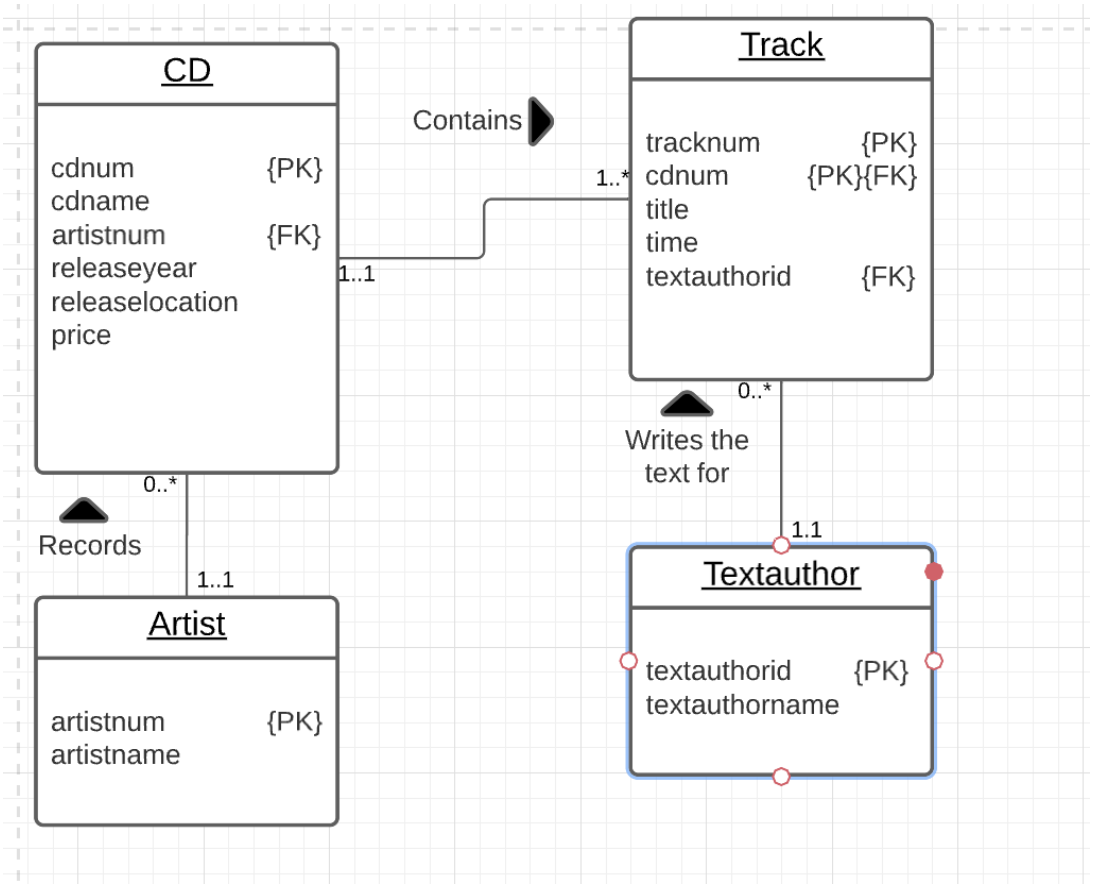
   Table Track:

   Note: In the table track I chose to add the column “title”, since it is difficult to register CD tracks without song titles. ;-) I also took out the column artistnum: It is specified in the

assignment text that it should be the same artist for an entire CD, then it is not needed here (finds artist through CD.artistnum).

- cdnum, tracknum -> **title,** time, ~~artistnum~~, textauthorid, textauthorname (*note:* title is new column, artistnum is removed)
- ~~cdnum -> artistnum~~ (*note:* artistnum is removed, so not relevant here anymore)
- textauthorid -> textauthorname

After normalization we have 4 tables: `CD, Track, Artist, Textauthor`



c) Write down an explanation of why your solution is at 1NF, 2NF, 3NF, and BCNF.

In you own words, write the explanation, based on the normalization rules. It should also refer to the determinants you have found. (Listed in an answer further up.)

1NF: The cells contain only one data element each. (I know this as they are a "database structure" already).

2NF: There are no dependencies on parts of the primary key. My solution has only one composite primary key (in Track) and the attributes in Track that are not part of the PK are all functionally dependent on the entire composite PK (tracknum and cdnum).

3NF: No column outside the primary key has a transitive dependency on the PK. However, the database structure had this problem before normalization, both regarding artistnum -> artistname and textauthorid -> textauthorname).

BCNF: All determinants are also candidate keys. The determinants are described above, and these are all primary keys (selected candidate keys) in the tables.

d) Upload your model to the forum if you would like to get feedback on it.

Kudos to those who dared to do this! :-)

Task 3 - Implementation in MySQL

Implement (write SQL for) the database you have modeled in task b, above.

Then enter a couple of CDs with corresponding tracks. (You can find data on google or come up with data yourself.)

```
-- SQL to create schema and tables:

DROP SCHEMA IF EXISTS musicarchive;

CREATE SCHEMA musicarchive;
USE musicarchive;

CREATE TABLE artist
(
	artistnum int NOT NULL AUTO_INCREMENT,
	artistname varchar(50) NOT NULL,
	CONSTRAINT artist_artistnum_pk PRIMARY KEY (artistnum)
);

CREATE TABLE cd
(
	cdnum int NOT NULL auto_increment,
	cdname varchar(50) NOT NULL,
	artistnum int NOT NULL,
	releaseyear smallint,
	releaselocation varchar(50),
	price float,
	CONSTRAINT cd_cdnum_pk PRIMARY KEY (cdnum),
	CONSTRAINT cd_artistnum_fk
		FOREIGN KEY (artistnum) REFERENCES artist(artistnum)
);

CREATE TABLE textauthor
(
	textauthorid int NOT NULL AUTO_INCREMENT,
	textauthorname varchar(50) NOT NULL,
	CONSTRAINT textauthor_textauthorid_pk
		PRIMARY KEY (textauthorid)
);

CREATE TABLE track
(
	cdnum int NOT NULL,
	tracknum int NOT NULL,
	title varchar(50) NOT NULL,
	time time NOT NULL,
	textauthorid int NOT NULL,
```

```
      CONSTRAINT track_tracknum_cdnum_pk PRIMARY KEY (tracknum, cdnum),
      CONSTRAINT track_cdnum_fk FOREIGN KEY (cdnum) REFERENCES cd(cdnum),
      CONSTRAINT track_textauthorid_fk FOREIGN KEY
            (textauthorid) REFERENCES textauthor(textauthorid)
);

-- SQL to insert data:

insert into artist (artistname) values
('AC/DC'),
('Prince'),
('Madonna');

insert into textauthor (textauthorname) values
('Angus Young, Malcolm Young'),
('Prince'),
('Madonna'),
('Reggie Lucas'),
(' Curtis Hudson, Lisa Stevens');

insert into cd values
(NULL, 'Black Ice', 1, 2008, 'internationally', NULL) ,
(NULL, '20ten', 2, 2010, 'UK and Ireland', NULL),
(NULL, 'Madonna', 3, 1983, NULL, NULL);

insert into track values
(1, 1, 'Rock \'n\' Roll Train', '00:04:21', 1),
(1, 2, 'Skies on Fire', '00:03:34', 1),
(1, 3, 'Big Jack', '00:03:57', 1),
(1, 4, 'Anything Goes', '00:03:22', 1),
(1, 5, 'War Machine', '00:03:09', 1),
(1, 6, 'Smash \'n\' Grab', '00:04:06', 1),
(1, 7, 'Spoilin\' for a Fight', '00:03:17', 1),
(1, 8, 'Wheels', '00:03:28', 1),
(1, 9, 'Decibel', '00:03:34', 1),
(1, 10, 'Stormy May Day', '00:03:10', 1),
(1, 11, 'She Likes Rock \'n\' Roll', '00:03:53', 1),
(1, 12, 'Money Made', '00:04:15', 1),
(1, 13, 'Rock \'n\' Roll Dream', '00:04:41', 1),
(1, 14, 'Rocking All the Way', '00:03:22', 1),
(1, 15, 'Black Ice', '00:03:25', 1);

insert into track values
(2, 1, 'Compassion', '00:03:57', 2),
(2, 2, 'Beginning Endlessly', '00:05:27', 2),
(2, 3, 'Future Soul Song', '00:05:08', 2),
(2, 4, 'Sticky Like Glue', '00:04:46', 2),
(2, 5, 'Act of God', '00:03:13', 2),
(2, 6, 'Lavaux', '00:03:03', 2),
(2, 7, 'Walk in Sand', '00:03:29', 2),
(2, 8, 'Sea of Everything', '00:03:49', 2),
(2, 9, 'Everybody Loves Me', '00:04:08', 2),
(2, 10, 'Laydown', '00:03:07', 2);

insert into track values
```

```
(3, 1, ' Lucky Star', '00:05:37', 3),
(3, 2, ' Borderline', '00:05:20', 4),
(3, 3, ' Burning Up', '00:03:45', 3),
(3, 4, ' I Know It', '00:03:47', 3),
(3, 5, ' Holiday', '00:06:10', 5),
(3, 6, ' Think of Me', '00:04:54', 3),
(3, 7, ' Physical Attraction', '00:06:39', 4),
(3, 8, ' Everybody', '00:04:57', 3);
```

Task 4 - Query in MySQL

Write a couple of queries against the database. E.g. :

- Extract all tracks from a CD.

  (*Note:* Here solved with a subquery but could have been solved with a JOIN as well.)

  ```
  select *
  from track
  where cdnum = (select cdnum
                 from cd
                 where cdname = 'Black Ice');
  ```

- Artist based on a given track.

  ```
  select artist.artistname
  from artist
  join cd on artist.artistnum = cd.artistnum
  join track on cd.cdnum = track.cdnum
  where track.title = 'Act of God';
  ```

- Extract relevant info for a track, based on the text author.

  ```
  select cd.releaseyear, cd.cdname, tr.title, ar.artistname,
         ta.textauthorname
  from cd
  join track tr on cd.cdnum = tr.cdnum
  join artist ar on cd.artistnum = ar.artistnum
  join textauthor ta on tr.textauthorid = ta.textauthorid
  where ta.textauthorname like '%Lucas%';
  ```

- Etc.

  Come up with your own queries for this one. :-)